Scalability Report: Exchange Matching Engine (ECE 568 Homework 4)

Yao-Hsuan Chang (yc658)
Ankit Raj (ar791)

Introduction

This report documents the scalability analysis of a multithreaded Exchange Matching Engine server. The objective of the scalability test is to measure how server performance (throughput and latency) scales with increasing CPU core counts, under concurrent XML-based client requests.

The engine supports creating accounts and symbols, placing buy/sell orders, executing matched trades, and querying or canceling orders. Matching follows a price-time priority policy. Atomic execution and thread safety are enforced using appropriate locks.

Experimental Setup

- **Server language**: C++17
- **Test client**: Python 3
- **Communication**: TCP socket + TinyXML2 for XML parsing
- **Machine**: ECE 568 Multi-Core VM
- **Cores tested**: 1, 2, and 4 cores (using taskset)
- **Server thread pool**: Equal to the number of assigned cores
- **Test case**: 100 order requests using XML format sent via concurrent client threads

---

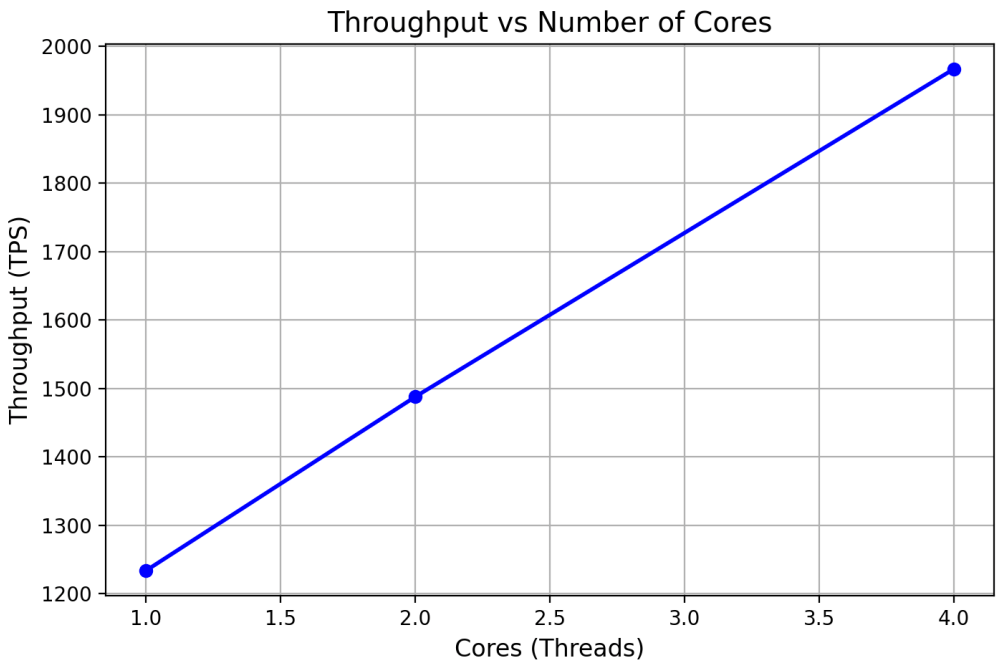
Methodology

- Each experiment was repeated 5 times and average values reported.
- Used taskset to bind the server to different numbers of CPU cores.
- For each core configuration, the client spawned a matching number of threads to saturate the server.
- Metrics measured:
    - Total execution time

- Throughput (transactions per second, TPS)
- Average latency per request (seconds)

Results and Graph

Below is the graph showing throughput as core count increases:



Raw data:

Cores	TPS	Total Time (s)	Avg Latency (s)
1	1233.99	0.081	0.001
2	1487.99	0.067	0.001
4	1966.82	0.051	0.002

Analysis

- **Throughput Improvement**: Throughput increases with the number of cores. From 1 to 4 cores, TPS improved by ~59%.

- **Latency**: Average latency remained low (~1-2ms) across all core counts, with slight increase at 4 threads due to parallel scheduling and queueing overhead.
- **Scalability**: The server scales well with more cores, benefiting from parallel request processing via a thread pool. However, locking contention in shared structures (e.g., accounts, order books) may become the next bottleneck beyond 4 cores.

Conclusion

The Exchange Matching Engine demonstrates good scalability up to 4 cores. Increasing the thread pool size along with core count allows parallel handling of concurrent requests. Proper locking ensures correctness while minimizing contention. Future optimization could involve finer-grained locks and batching. These enhancements are expected are to further boost performance, especially as the system scales to support more cores.